

Entrega 2 – Sistemas Operacionais e Computação em Nuvem

Integrantes:

Antônio Petri RA: 24026144

Leonardo Santos da Silva RA: 24026495

Lucas de Lima Gutierrez RA: 24026013

Vitor Kenzo Kanashiro RA: 24026113

SOBRE O PROJETO

O Projeto Lideranças Empáticas é um sistema composto por três módulos principais: um aplicativo mobile, um backend FastAPI e um dashboard de visualização em Dash/Python. Para garantir que todos os serviços sejam executados de forma consistente e isolada, independente do ambiente, o projeto utiliza Docker como tecnologia de containerização.

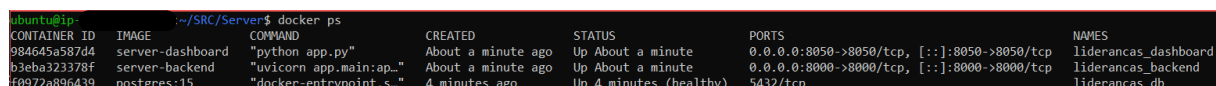
Cada módulo possui seu próprio Dockerfile, e a orquestração entre os serviços é feita com Docker Compose, implantado na instância EC2 da AWS configurada na Entrega 1.

ARQUITETURA DOS SERVIÇOS

O ambiente é composto por três containers que se comunicam através da rede interna appnet:

- db — banco de dados PostgreSQL 15, acessível apenas internamente
- backend — servidor FastAPI/Uvicorn, exposto na porta 8000
- dashboard — aplicação Dash/Python, exposta na porta 8050

O banco de dados aguarda a confirmação do healthcheck antes de permitir que o backend suba, evitando erros de conexão na inicialização.

A terminal window showing the command 'docker ps' and its output. The output is a table with columns: CONTAINER ID, IMAGE, COMMAND, CREATED, STATUS, PORTS, and NAMES. There are three containers listed: 'l1derancas_dashboard', 'l1derancas_backend', and 'l1derancas_db'.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
984645a587d4	server-dashboard	"python app.py"	About a minute ago	Up About a minute	0.0.0.0:8050->8050/tcp, [::]:8050->8050/tcp	l1derancas_dashboard
b3eba323378f	server-backend	"uvicorn app.main:ap..."	About a minute ago	Up About a minute	0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp	l1derancas_backend
f0972a896439	postgres:15	"docker-entrypoint.s..."	4 minutes ago	Up 4 minutes (healthy)	5432/tcp	l1derancas_db

Figura 1 – Containers em execução na instância EC2

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
postgres:15	a116b2156a68	445MB	0B	U
server-backend:latest	abd80add6841	431MB	0B	U
server-dashboard:latest	5eaf4d4abcbb	429MB	0B	U

Figura 2 – Images na instância EC2

DOCKERFILES

Backend (Server/Dockerfile)

Responsável por construir a imagem do servidor FastAPI. Parte da imagem python:3.11-slim, instala o compilador gcc para suportar bibliotecas com extensões nativas (como psycpg2) e executa o Uvicorn escutando em todas as interfaces de rede.

```

Server > Dockerfile > ...
1  FROM python:3.11-slim
2
3  WORKDIR /app
4
5  RUN apt-get update && apt-get install -y \
6      gcc \
7      && rm -rf /var/lib/apt/lists/*
8
9  COPY requirements.txt .
10 RUN pip install --no-cache-dir -r requirements.txt
11
12 COPY ./app /app/app
13
14 CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
15

```

Código 1 – Dockerfile do Backend

Dashboard (Dashboard/Dockerfile)

Constrói a imagem do dashboard de visualização. Segue a mesma estrutura base, copiando as dependências antes do código-fonte para aproveitar o cache de camadas do Docker e agilizar reconstruções.

```

Dashboard > Dockerfile > ...
1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7
8 COPY . .
9
10 EXPOSE 8050
11
12 CMD ["python", "app.py"]
13

```

Código 2 – Dockerfile do Dashboard

DOCKER COMPOSE

O arquivo docker-compose.yml define e orquestra os três serviços. Os principais recursos utilizados são:

- healthcheck no banco de dados, garantindo que o PostgreSQL esteja pronto antes do backend iniciar
- rede interna appnet, isolando a comunicação entre os containers
- volume persistente postgres_data, mantendo os dados mesmo após reinicialização dos containers
- variáveis de ambiente carregadas via arquivo .env, mantendo as credenciais fora do código
- restart: always, reiniciando os serviços automaticamente em caso de falha

```

Server > docker-compose.yml
  Run All Services
  Run Service
1 services:
2   db:
3     image: postgres:15
4     container_name: liderancas_db
5     restart: always
6     environment:
7       POSTGRES_DB: ${POSTGRES_DB}
8       POSTGRES_USER: ${POSTGRES_USER}
9       POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
10    volumes:
11      - postgres_data:/var/lib/postgresql/data
12    networks:
13      - appnet
14    healthcheck:
15      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
16      interval: 5s
17      timeout: 5s
18      retries: 10
19

```

```

20 backend:
21     build: .
22     container_name: liderancas_backend
23     restart: always
24     env_file:
25         - .env
26     depends_on:
27         db:
28             condition: service_healthy
29     volumes:
30         - ./app:/app/app
31     ports:
32         - "8000:8000"
33     networks:
34         - appnet
35

```

```

36 dashboard:
37     build: ../Dashboard
38     container_name: liderancas_dashboard
39     restart: always
40     env_file:
41         - ../Dashboard/.env
42     environment:
43         - SERVER_URL=http://liderancas_backend:8000
44     volumes:
45         - ../Dashboard:/app
46     ports:
47         - "8050:8050"
48     networks:
49         - appnet
50     depends_on:
51         - backend
52

```

```

53 volumes:
54     postgres_data:
55
56 networks:
57     appnet:
58

```

Código 3 – Docker Compose de produção

DIFICULDADES ENCONTRADAS

Problemas enfrentados:

- Erro de conexão do backend com o banco de dados por inicialização prematura
- Compilação de bibliotecas nativas Python sem o gcc instalado na imagem

- Comunicação entre os containers dashboard e backend usando endereços corretos na rede interna
- Configuração do arquivo .env com as variáveis corretas no servidor

Soluções aplicadas:

- Implementação de healthcheck com condition: service_healthy
- Adição do gcc no Dockerfile do backend via apt-get
- Uso da rede appnet com resolução por container_name
- Criação do arquivo .env diretamente no servidor via nano

RESULTADOS OBTIDOS

- Dockerfiles funcionais para o backend e o dashboard
- Docker Compose com os três serviços iniciando na ordem correta
- Projeto em execução na instância EC2 com docker compose up -d --build
- Backend acessível na porta 8000 e dashboard na porta 8050
- Dados do banco persistidos em volume após reinicialização dos containers

```

root@ubuntu:~# docker compose up -d --build
[+] Building 16.8s (18/18) FINISHED
=> [backend internal] load build definition from Dockerfile
=> => transferring dockerfile: 333B
=> [dashboard internal] load metadata for docker.io/library/python:3.11-slim
=> [backend internal] load .dockerignore
=> => transferring context: 2B
=> [dashboard 1/5] FROM docker.io/library/python:3.11-slim@sha256:6d85378d88a19cd4d76879817532d62232be95757cb45945a99fec8e8884b9c2
=> [backend internal] load build context
=> => transferring context: 104.75kB
=> CACHED [dashboard 2/5] WORKDIR /app
=> CACHED [backend 3/6] RUN apt-get update && apt-get install -y gcc && rm -rf /var/lib/apt/lists/*
=> CACHED [backend 4/6] COPY requirements.txt .
=> CACHED [backend 5/6] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [backend 6/6] COPY ./app /app/app
=> [backend] exporting to image
=> => exporting layers
=> => writing image sha256:abd80add6841450ee8df3514770ad6f03870f82b3d7872f2a248f8eb2f71b99
=> => naming to docker.io/library/server-backend
=> [dashboard internal] load build definition from Dockerfile
=> => transferring dockerfile: 200B
=> [dashboard internal] load .dockerignore
=> => transferring context: 2B
=> [dashboard internal] load build context
=> => transferring context: 491.58kB
=> CACHED [dashboard 3/5] COPY requirements.txt .
=> CACHED [dashboard 4/5] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [dashboard 5/5] COPY . .
=> [dashboard] exporting to image
=> => exporting layers
=> => writing image sha256:5eaf4dd4abcb42f3ebb6a69014d4cb564ee46b121421f929a2b982c2a6e91fc6
=> => naming to docker.io/library/server-dashboard
[+] Running 3/3
✔ Container liderancas_db Healthy 0.7s
✔ Container liderancas_backend Running 0.0s
✔ Container liderancas_dashboard Running 0.0s

```

Figura 2 – Deploy realizado na instância EC2

SERVIÇOS EM EXECUÇÃO

Com os containers em execução na instância EC2, os três serviços ficam acessíveis via navegador pelo IP público da máquina. O Docker garante que cada serviço suba isolado, na porta correta, e se comunique internamente com os demais.

Backend — Documentação da API (54.90.66.229:8000/docs)

O container do backend expõe automaticamente a documentação interativa da API (Swagger UI) na rota /docs. Isso confirma que o FastAPI está rodando corretamente dentro do container e aceitando conexões externas na porta 8000.

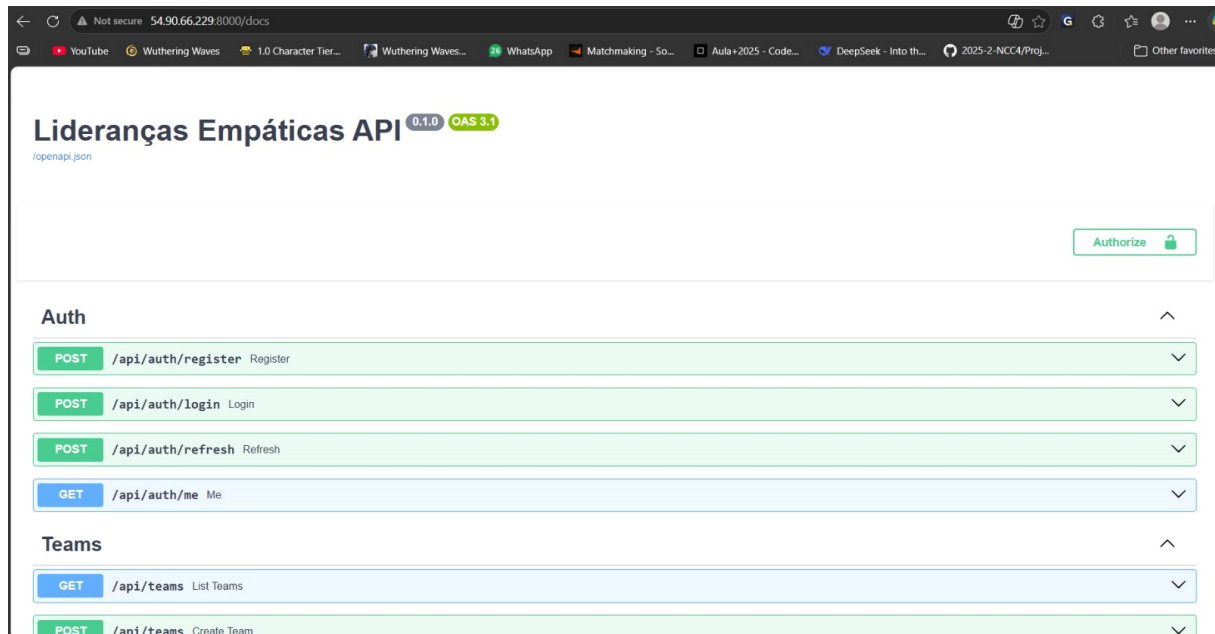


Figura 3 – Swagger UI do backend FastAPI rodando na EC2

Dashboard — Tela de Login (54.90.66.229:8050/login)

O container do dashboard sobe uma aplicação web completa com autenticação. A tela de login é o primeiro ponto de acesso, servida pelo Dash/Python rodando dentro do container na porta 8050.

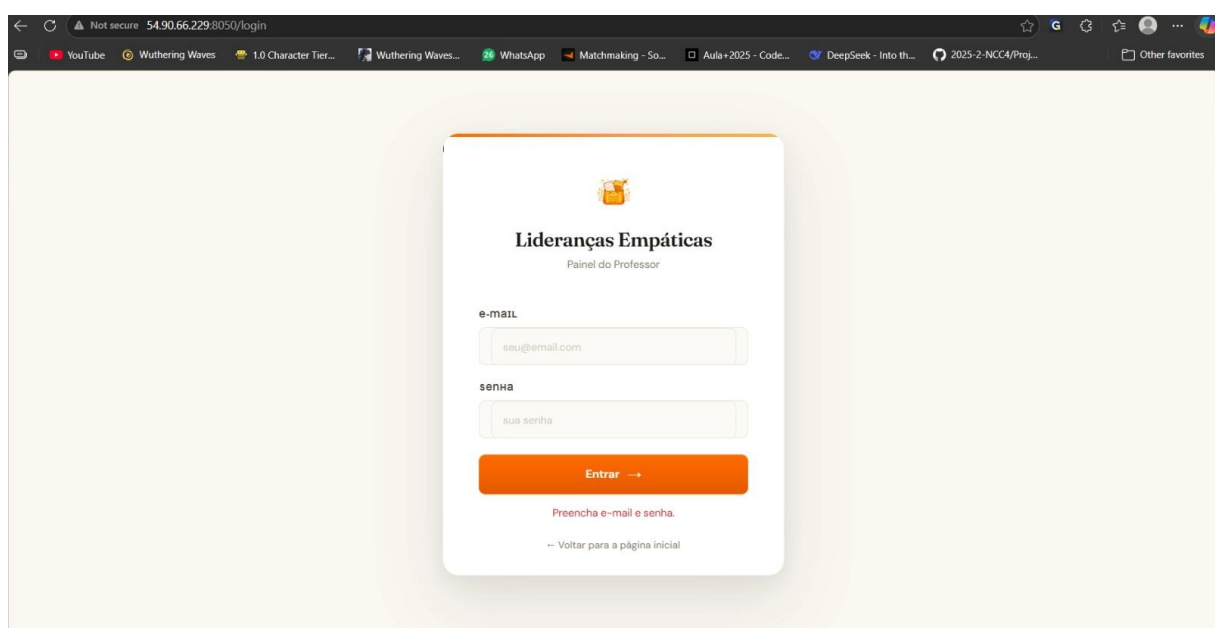


Figura 4 – Tela de login do Dashboard acessível via EC2

Dashboard — Painel Admin (54.90.66.229:8050/dashboard)

Após autenticar, o painel admin exibe os dados coletados pelo projeto: total de kg arrecadados pelo app e pela câmera, valor estimado, metas e gráficos por categoria. Todos esses dados vêm do container do backend via requisições internas na rede appnet.

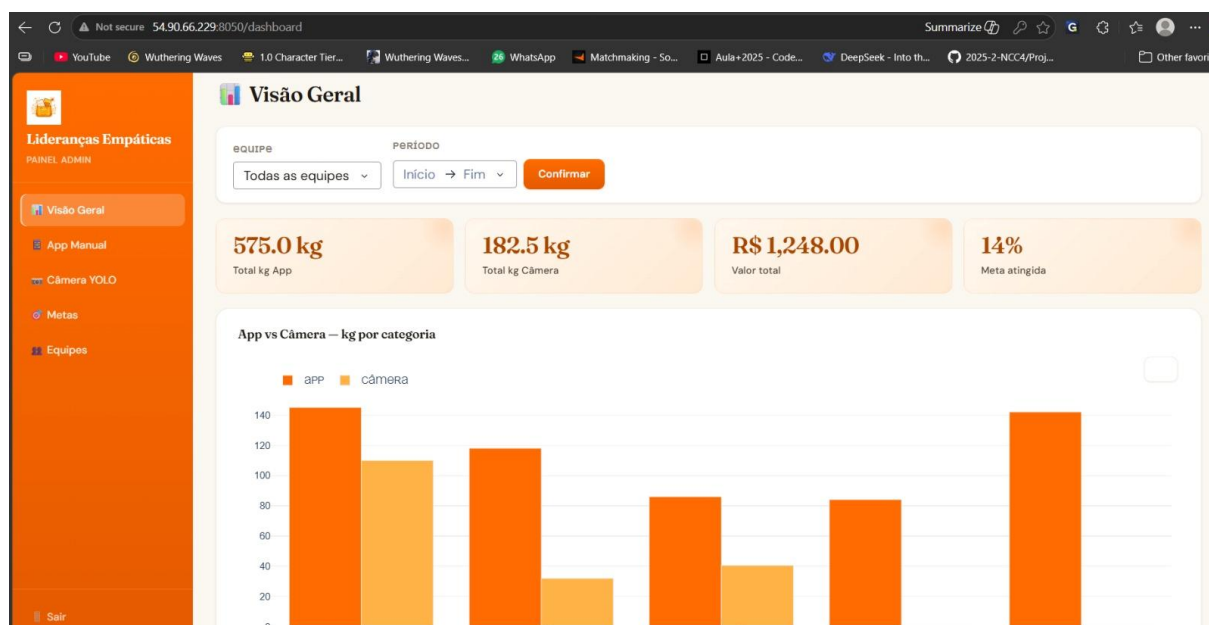
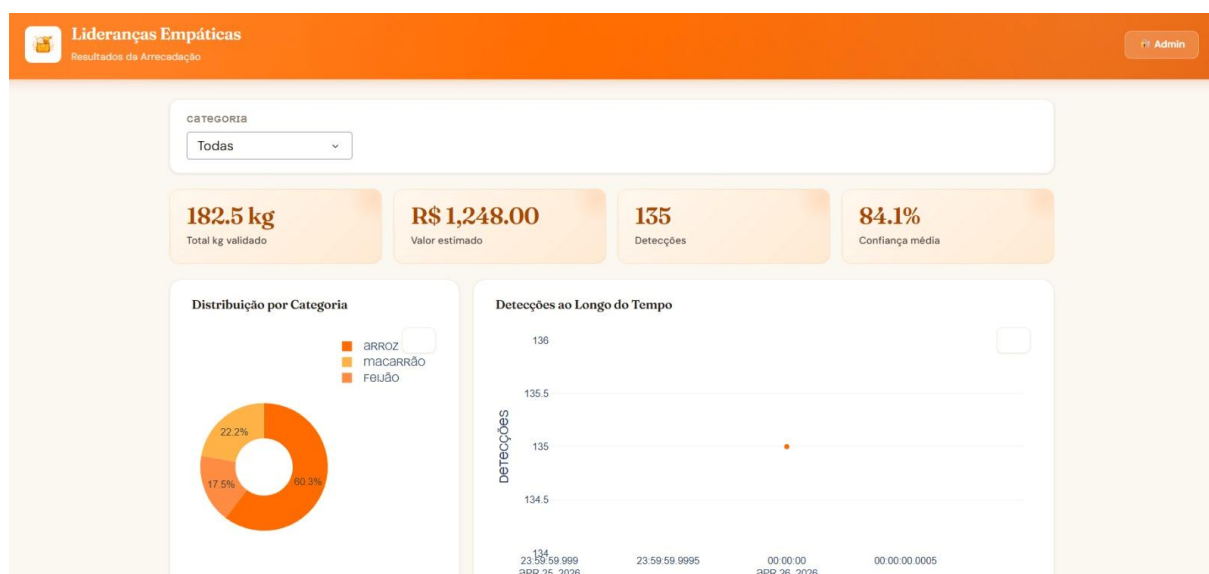


Figura 5 – Painel admin do Dashboard com dados em tempo real

Dashboard — Resultados da Arrecadação

O dashboard também exibe os resultados da detecção automática por câmera, mostrando o total de kg validados, valor estimado, número de detecções e a confiança média do modelo. Esses dados são processados pelo backend e consultados em tempo real pelo dashboard através da rede interna Docker.



CONCLUSÃO

A atividade permitiu aplicar na prática os conceitos de isolamento de processos e virtualização estudados na disciplina. Com o Docker, foi possível garantir que o ambiente de desenvolvimento e o servidor em nuvem se comportem de forma idêntica, eliminando problemas de compatibilidade entre máquinas.

A containerização do Projeto Lideranças Empáticas é a base que permitirá o deploy contínuo do sistema ao longo do semestre, com os três serviços funcionando de forma integrada e reproduzível na instância EC2 da AWS.